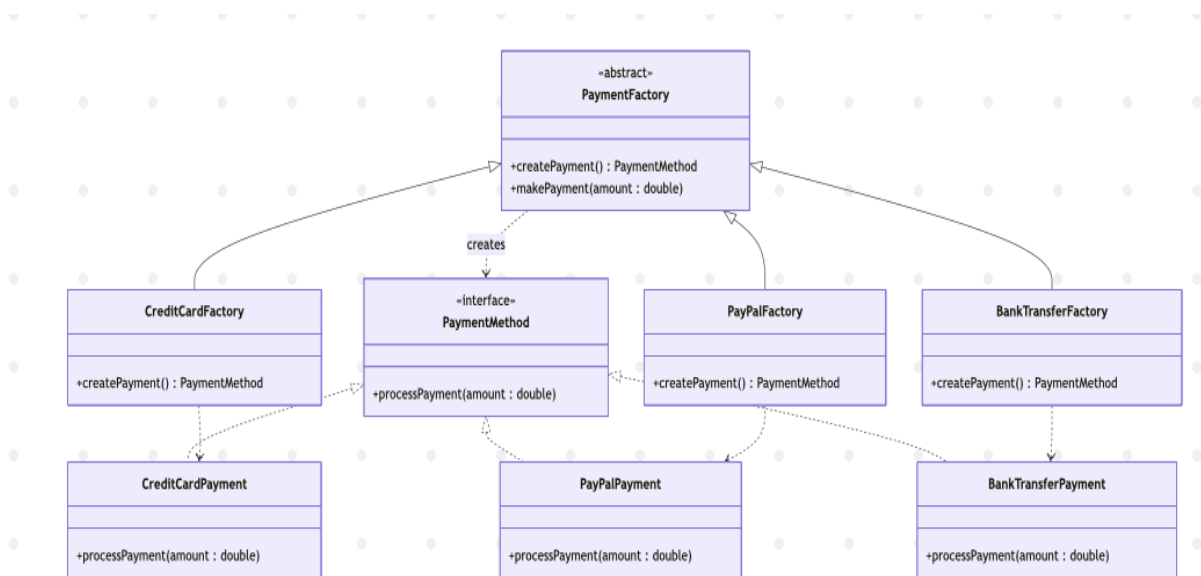


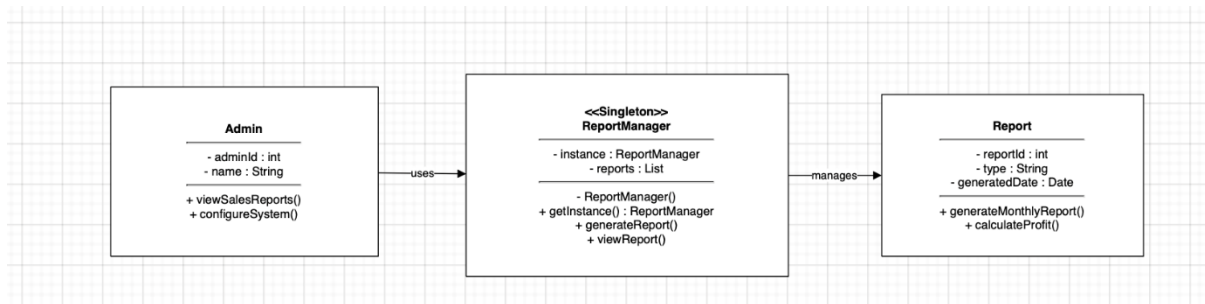
Henri Shehu & Alisa Dhima



Factory Pattern — Justification & Class Diagram

For our project, we picked the Factory Method pattern for handling payments in the Tourist Agency System. We chose this because the system supports multiple payment methods, including credit card, PayPal, and bank transfer, where each payment type requires different processing logic. Using this pattern helps us decouple the code, meaning the part of the system that processes payments does not need to know the exact details of how each payment method is created. As shown in our Class Diagram, we introduced an abstract **PaymentFactory** with a `createPayment()` factory method, while concrete factories such as **CreditCardFactory**, **PayPalFactory**, and **BankTransferFactory** are responsible for creating their corresponding **PaymentMethod** products. This allows the Customer or BookingAgent to simply work with the **PaymentMethod** interface without depending on specific payment classes. The pattern also makes the system much easier to scale because new payment methods can be added in the future by creating a new factory and product pair without modifying the existing code. This keeps the application maintainable, flexible, and aligned with standard object-oriented design principles.

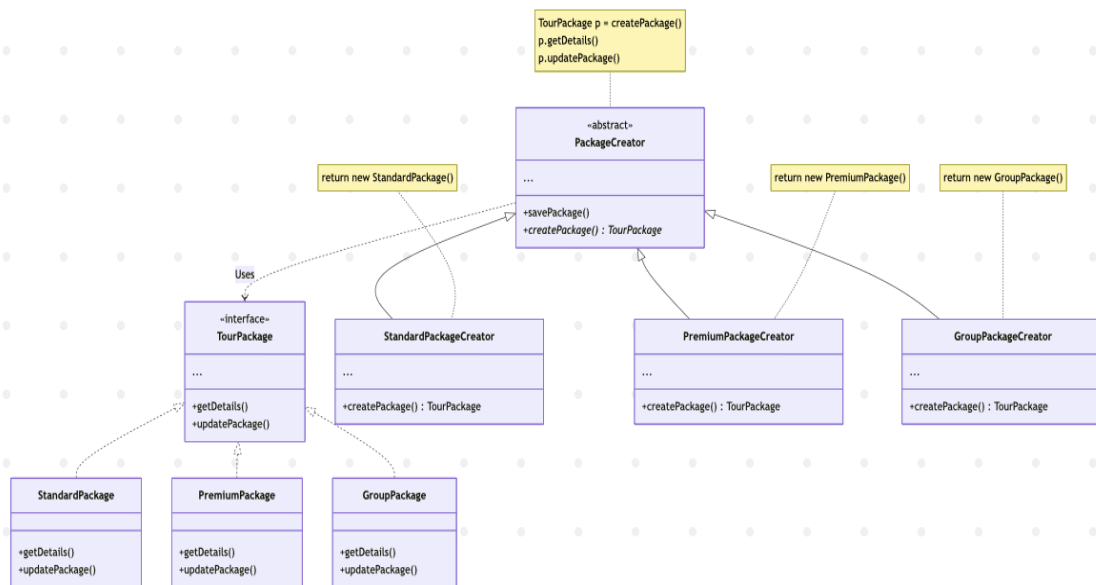
Amar Kruja & Lumturi Bedalli



Singleton Pattern — Justification & Class Diagram

For our project, we picked the Singleton pattern for managing reports in the tourist agency system. We chose this pattern because report generation is a system-wide operation that should be controlled from a single centralized point. If multiple instances of the ReportManager class were allowed, different parts of the system — such as the Admin viewing sales reports or the FinanceAgent generating financial reports — could produce duplicated or inconsistent report data. As shown in our Class Diagram, the ReportManager class is designed with a private constructor and a static getInstance() method, ensuring that only one instance of the class can exist throughout the application. This guarantees that all report-related operations are handled through the same shared object, maintaining consistency and reliability across the system. The Singleton pattern also improves resource management and keeps the report handling logic centralized, making the application easier to maintain and control.

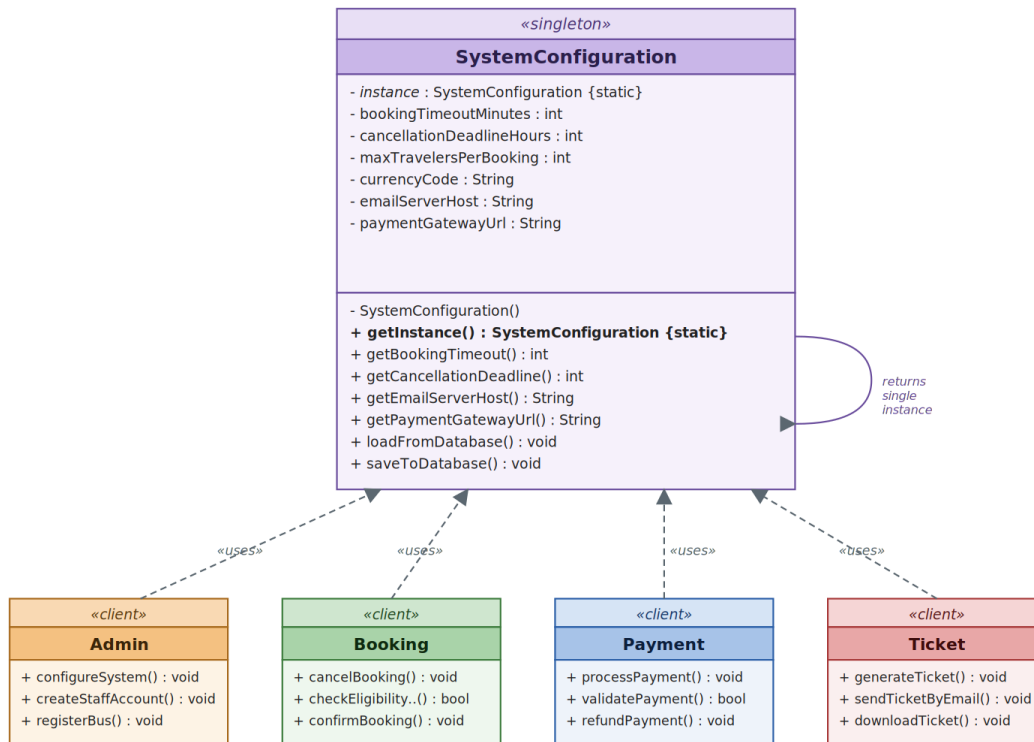
Emri Xhaferi & Era Selmanaj



Factory Pattern — Justification & Class Diagram

For our project, we picked the Factory Method pattern for creating tour packages. We chose this because, in BR_TM_21, the Operations Agent has to manage three different types of tours (Standard, Premium, and Group). Using this pattern helps us decouple the code, meaning the part of the system that saves the tour doesn't need to know the specific details of how each package is built. As shown in our Class Diagram, this makes it way easier to scale. If we need to add more tour categories in the future, we don't have to change the main creation logic; we just add a new creator class. This keeps our code maintainable and follows the standard design principles we learned.

Amanda Stafasani & Enian Xhixha



Singleton Pattern — Justification & Class Diagram

How the diagram reads:

- **SystemConfiguration** is the **Singleton** — it holds a private static reference to its own single instance and exposes it through the **getInstance()** method
- The constructor is marked private (**– SystemConfiguration()**) so no client can use *new* to build another instance
- **Admin**, **Booking**, **Payment**, and **Ticket** are the **clients** — they all reach the same configuration object through **SystemConfiguration.getInstance()** instead of holding their own copies
- The static field *instance* and the static method **getInstance()** are shown in bold/italic with the {static} tag to emphasise that they belong to the class itself, not to any object

The dashed arrows with open arrowheads represent *dependency* («uses»): each client class depends on SystemConfiguration but does not own it. The looped arrow on the Singleton itself indicates that getInstance() returns a reference to the same single instance every time it is called.

The Singleton pattern is applied to the SystemConfiguration class. Activity diagram BR_TM_25 (Configuration of system settings) shows the Administrator accessing one settings module, updating configurations, and saving them, after which those settings govern behavior across the entire system. Without the Singleton pattern, different parts of the system could load their own copies of these settings, producing inconsistent rules — for example, one booking applying a different cancellation deadline than another, or two payment requests using different gateway URLs. By making SystemConfiguration a Singleton, the cancellation deadline used by Booking.checkEligibilityForCancel() in BR_TM_11, the payment gateway URL used by Payment.processPayment() in BR_TM_07, and the email server host used by Ticket.sendTicketByEmail() in BR_TM_09 all read from the same authoritative source. The pattern guarantees a single point of truth for system-wide configuration and a global, controlled point of access to it.